Check for updates

ACM DIGITAL LIBRARY | Association for Computing Machinery | acm open

Latest updates: https://dl.acm.org/doi/10.1145/3446132.3446165

RESEARCH-ARTICLE

Reinforced Evolutionary Algorithms for Game Difficulty Control

GUANGWU CUI, Xiangtan University, Xiangtan, Hunan, China

RUIMIN SHEN

YINGFENG CHEN

JUAN ZOU, Xiangtan University, Xiangtan, Hunan, China

SHENGXIANG YANG, De Montfort University, Leicester, Leicestershire, U.K.

CHANGJIE FAN

View all

Open Access Support provided by:

Xiangtan University

De Montfort University

PDF Download
3446132.3446165.pdf
17 February 2026
Total Citations: 1
Total Downloads: 196

Published: 24 December 2020

Citation in BibTeX format

ACAI 2020: 2020 3rd International Conference on Algorithms, Computing and Artificial Intelligence
December 24 - 26, 2020
Sanya, China

ACAI '20: Proceedings of the 2020 3rd International Conference on Algorithms, Computing and Artificial Intelligence (December 2020)
https://doi.org/10.1145/3446132.3446165
ISBN: 9781450388115

Reinforced Evolutionary Algorithms for Game Difficulty Control

Guangwu Cui*
cuigwu@gmail.com
Xiangtan University
Xiangtan, Hunan, China

Ruimin Shen
NetEase
Shenzhen, China
ruimin0shen@gmail.com

Yingfeng Chen
NetEase Fuxi
Hangzhou, China
chenyingfeng1@corp.netease.com

Juan Zou
Xiangtan University
Xiangtan, Hunan, China
zoujuan@xtu.edu.cn

Shengxiang Yang
De Montfort University
Leicester, East Midlands, England
syang@dmu.ac.uk

Changjie Fan
NetEase Fuxi
Hangzhou, China
fanchangjie@corp.netease.com

Jinghua Zheng
Xiangtan University
Xiangtan, Hunan, China
jhzheng@xtu.edu.cn

ABSTRACT

In the field of game designing, artificial intelligence is used to generate responsive, adaptive, or intelligent behaviors primarily in Non-Player-Characters (NPCs). There is a large demand for controlling game AI since a variety of players expect to be provided NPC opponents with appropriate difficulties to improve their game experience. However, to the best of our knowledge, a few works are focusing on this problem. In this paper, we firstly present a Reinforced Evolutionary Algorithm based on the Difficulty-Difference objective (REA-DD) to the DLAI problem, which combines reinforcement learning and evolutionary algorithms. REA-DD is able to generate the desired difficulty level of game AI accurately. Nonetheless, REA can only obtain a kind of game AI in each run. To improve efficiency, another algorithm based on Multi-objective Optimization is proposed, regarded as RMOEA-DD, which obtains DLAI after one run. Experiments on the game Pong from ALE and apply on a commercial game named *The Ghost Story* to show that our algorithms provide valid methods to the DLAI problem both in the term of controlling accuracy and efficiency.

CCS CONCEPTS

• **Computer systems organization** → *Neural networks*; • **Computing methodologies** → **Intelligent agents**.

KEYWORDS

Game AI, Difficulty levels, Evolutionary algorithms, Reinforcement learning

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ACAI 2020, December 24–26, 2020, Sanya, China
© 2020 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-8811-5/20/12…$15.00
https://doi.org/10.1145/3446132.3446165

ACM Reference Format:
Guangwu Cui, Ruimin Shen, Yingfeng Chen, Juan Zou, Shengxiang Yang, Changjie Fan, and Jinghua Zheng. 2020. Reinforced Evolutionary Algorithms for Game Difficulty Control. In *2020 3rd International Conference on Algorithms, Computing and Artificial Intelligence (ACAI 2020), December 24–26, 2020, Sanya, China.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3446132.3446165

1 INTRODUCTION

Over the past few decades, video games have gained increasing popularity due to their entertainment. Artificial intelligence (AI) techniques have been widely used to design various Non-Play-Character (NPC) to satisfy the interest of players. The study of designing various NPC for the field of games in this paper, refers to game AI. In many games, it is necessary to develop a suitable game AI and adopt it to entertain players. However, it is not straightforward to design such a game AI. A suitable game AI which is comparable to level of human player in the game. The over-high difficulty level of game AI will make the players lose confidence. On the contrary, the over-low difficulty level will make the players lose interest. Generally, it requires lots of artificial experience from professional game developers to design a game AI, which leads to large consumption of time and cost. Thus, to relieve the burden of game designers, an automatic method to generate a suitable game AI for players of different levels is required. However, to the best of our knowledge, there are few works focusing on this problem. This aim of this paper is to solve the problem of generating Different difficulty Levels of game AI (DLAI). Although few work is direct to the DLAI problem, reinforcement learning (RL) provides a potential way to obtain a higher difficulty level of game AI. RL has achieved excellent results in fields of the game AI, such as the role-playing game, the board game and the shooter game. Meanwhile, the evolutionary algorithm (EA)[11] is also competitive in this field, which achieve good results in a variety of games like Atari games and the game of Go. Both of EA and RL have their own strong points, the combination of them may take respective advantages. This paper will present two reinforced evolutionary algorithms (REAs) to control DLAI more accurately by difficulty difference as the objective function. In REAs, the main concern is the accuracy of generating

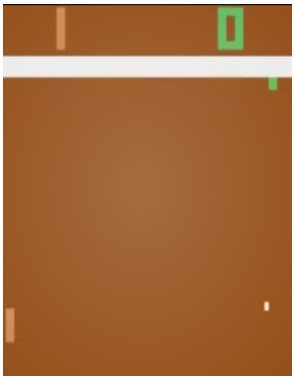


Figure 1: A screenshot of Pong.

DLAI. REA-DD only generates a kind of difficulty-level game AI the experiment in one run. Consequently, to improve the efficiency of the algorithm, we will change the objective number of REA-DD and present multi-objective REA-DD, named RMOEA. The algorithms are as follows:

- In REA-DD, the difficulty difference is used to build the fitness function. Since many games can be solved by approximate modeling into MDPs and RL can learn efficiently in MDPs [14], RL is used into REA-DD. REA-DD can offer DLAI for game adjustment in DDA.
- In RMOEA-DD, we adopt two objective functions that use the value of binding of the difficulty as the target. We used the crowed distance operator from NSGA-II [3] to maintain the distribution and diversity of the population, and used distribution and diversity to reduce the total difficulty difference.

Our contributions are on four major points. First, we provide a framework based on RL and EA to the DLAI problem. Second, we propose REA-DD, which not only can obtain powerful game AI, but also effectively control game AI's ability to delight various players. Third, we introduce a multi-objective algorithm based on REA-DD, called RMOEA-DD, which can save a lot of computational costs while effectively controlling DLAI. Fourth, our algorithms experiment on the game *Pong* shown in figure 1 and have been successfully applied to the game named *The Ghost Story* (Ghost) and achieved good results.

2 RELATED WORK

Reinforcement Learning. Recently, reinforcement learning has been undergoing extraordinary development and prosperity. AI agents have the ability beyond humans in some games. For example, the game AI agent AlphaGo developed by DeepMind defeated human player Lee Sedol who had won the champion in the game of Go for 18 times [12] and the game AI agent OpenAI Five developed by OpenAI gained a victory in the game Dota 2 playing with the world champion team OG [9]. RL aims to find the actions which yield the highest return by trying them [15]. The target of most RL is to maximize a cumulative reward in some way. The main technologies in RL can be classified into value-based techniques and policy-based techniques. The value-based RL is to learn a value function which represents how good an action is in the current state, such as Q-learning [20], SARSA [18], and DQN [19]. The policy-based RL is to directly parametrize the policy, which is to construct an evaluation mechanism to learn an action of agent [16]. The most impressive ones are the actor-critic (AC) [6] algorithm and the asynchronous advantage actor-critic (A3C) [8] algorithm, which combines the advantages of both policy-based and value-based techniques.

Evolutionary Algorithm. The evolutionary algorithm uses ideas from biological evolution [1]. EA has a good ability to explore due to its population mechanisms. In general, high-quality individuals from the population produce high-quality offspring in some forms (e.g., crossover operator and mutation operator). Individual quality is evaluated according to a fitness value which is calculated in some way, such as a fitness value obtained by interacting with the environment. EA is a non-gradient optimization algorithm that is good at solving complex problems. In the field of games, EA is used to evolve the neural network and was applied to predator/prey multi-character computer games [21]; The real-time neuroevolution of augmenting topologies method as an AI evolves increasingly its complexity of neural networks in the NERO video game [13]. The multiple objective evolutionary algorithm (MOEA) is an extension of EA [2]. It has a set of optimal solutions (referred to as Pareto front in objective space) in MOEA. Among them, the most representative algorithms are NSAG-II [3], SPEA-2 [23], and MOEA/D [22].

Game Difficulty Control. There are few studies on DLAI, but some still related researches to be found, such as dynamic difficulty adjustment (DDA). DDA is a method of automatically modifying a game's features, behaviors, and scenarios in real-time [24]. In DDA, dynamic scripts were used to online adjust the level of difficulty of the game AI [10] early times. The adaptive behavior-based AI uses ideas from RL and evolutionary computation for dynamic game difficulty scaling [17]. SARSA (λ) was used to provide the game AI of appropriate levels by storing a historical Q-table [4]. Monte-Carlo tree search was used to make dynamic difficulty adjustments in fighting games [5]. In DDA, the research focus to adjust the game difficulty online, but the premise is to have different levels of difficulty. Thus, it is also necessary to research the DLAI problem.

3 BACKGROUND

3.1 Reinforcement Learning

An MDP can be defined with a tuple like $< S, A, T, R, \gamma >$, where S is a finite set of states; A is a finite set of actions; $T_a(s, s') = Pr(s_{t+1} = s' | s_t = s, a_t = a)$ is the transition probability function that action a in state s at time t will lead to state s' at time $t + 1$; $R_a(s, s')$ is the immediate reward function gained after $T_a(s, s')$, and discount factor $\gamma \in [0, 1]$ decides the importance of future rewards.

In RL, the agent learns a policy by interacting with an environment. At each time step t, an agent chooses an action a_t, according to a policy π from the observation of state s_t of the current environment, and observes a reward r_t. In RL, the target of agent is to find a policy to maximize the expected discounted rewards $G_t = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}$, where T is the terminal time.

In the AC algorithm, the policy can be defined as $\pi_\theta(a|s)$ where the parameters θ is updated by gradient ascent on $\mathbb{E}_\theta[G_t]$ in the

direction $\nabla \log \pi_\theta(a_t|s_t)G_t$. In general, to reduce the variance of the estimate, we add a baseline $b_t(s_t)$, and it can be replaced with $V^\pi(s_t)$ in learning estimate of the value function. Thus the gradient is $\nabla \log \pi_\theta(a_t|s_t)(G_t - V^\pi(s_t))$. To improve the training speed, the asynchronous method is introduced in A3C [8] which used multiple threads to take samples from different training environments. This makes the distribution of samples more uniform, which is more conducive to the training of neural networks. At the same time, A3C's parallel computing is similar to parallelize the individual in EA. For this reason, to show the effectiveness of RMOEA compared with A3C.

3.2 Evolutionary Algorithm

In EA, it consists of four major elements: the coding of individuals, the design of fitness functions, genetic operations (selection, crossover, and mutation), and environmental selection. Compared to EA, MOEA is to solve multi-objective optimization problem. Multi-objective optimization problem can be defined by:

$$\max F(x) = (f_1(x), f_2(x), \cdots, f_m(x))^{\mathrm{T}} \tag{1}$$

$$s.t. \qquad F(x) \in \mathbb{R}^m, x \in \Omega, \tag{2}$$

where $x = (x_1, x_2, \cdots, x_m)^{\mathrm{T}}$ is a vector of decision variables in the decision space ω; $\mathbb{R}^m$ is an objective space, and $F : \omega \rightarrow \mathbb{R}^m$ consists of objective functions. The F-function is used to obtain an objective value (F-value) in $\mathbb{R}^m$. The optimal F-value can be referred as Pareto front (PF):

$$PF = \{F(x) \in \mathbb{R}^m \mid x \in PS\}, \tag{3}$$

where Pareto Set (PS) is the Pareto optimal set in $\mathbb{R}^m$. Given two individuals x_1 and x_2, x_1 Pareto dominance x_2, the relative of Pareto dominance is defined as below:

$$\forall i, f_i(x_1) \succeq f_i(x_2) \quad \&\& \quad \exists j, f_j(x_1) \succ f_j(x_2), \tag{4}$$

where $i, j \in \{1, 2, \cdots, m\}; x_1, x_2 \in \Omega$.

In MOEA, the two most important points are to obtain the non-dominated population and using the environment selection operator to maintain the population distribution. In this paper, we only need to consider the diversity and distribution of the population.

4 METHODOLOGY

The complicate artificial neural network used to encode the population's genes will result in complex population's genes in the EA. In general, complex genes accompany potential individuals which required learn or other methods to show their ability, before evaluating by function. To avoid a large number of potential individuals to eliminate, the learning operator is introduced in the EA. Many games can be solved by approximate modeling into MDPs. Thus, RL is used to combine with EA. From another perspective, the crossover and mutation operators in the evolutionary process did not make use of the information available during the game in the EA. For the reason, the efficiency of the exploitation of EA will be relatively low in solving the problem. After introducing RL, the algorithm may make better use of the information during the game.

This section describes the REAs based on RL and EA. The main difference is that REA-DD and RMOEA-DD have different optimization problems in EA. DLAI can be obtained through different

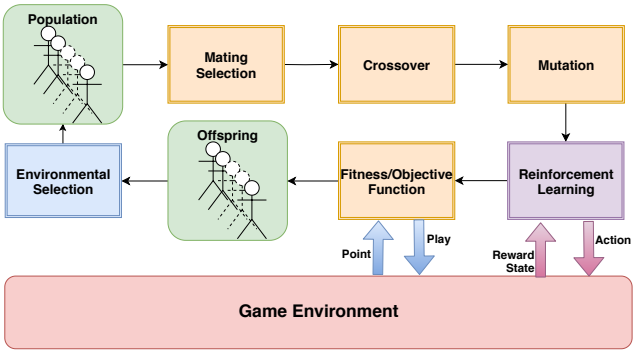


Figure 2: An illustration of the flowchart of REA-DD/RMOEA-DD to simulate the process of the population's evolution in the game environment. Compared to EA, the offspring will be trained by reinforcement learning after producing by the crossover operator and mutation operator, and the fitness or objective value of the individual is obtained by interacting with the environment of the game.

targets in REA-DD. RMOEA-DD may generate effectively many kinds of levels with lower cost compared to REA-DD. In REAs, AC, A3C and deep deterministic policy gradient (DDPG) [7] could be used to train the individual. This paper takes AC as an example. Figure 2 shows the flowchart of the REA-DD/RMOEA-DD on the game task. In the algorithms, each individual is encoded with an artificial neural network in the population. The genes are the weight of the individual. First, the individual is selected into genetic operators by tournament selection operators from the population. Then, the individual to reproduce uses genetic operators (crossover, mutation). The crossover operator generates a new individual by swapping weights of two individuals. The mutation operator is to change some weights of the individual generating by the crossover operator in some way. Every newly generated individual uses RL to interact with the environment to train itself. It forms offspring after using the fitness function or objective function interacting with the environment (such as obtained point from the environment after playing the game) to evaluate itself.

4.1 REA-DD

The purpose of REA-DD to the DLAI problem, at the same time, to control its difficulty more accurately. The main processes of REA-DD are shown in Algorithm 1. Each individual encodes with the artificial neural network which consists s of actor-network and critic-network. Typically, individuals are randomly initialized in a population, trained by RL (AC) to update their parameters θ. Then the individual is evaluated with a fitness function which defined by the formula:

$$f = -|\frac{wgc}{tgc} - tg|, \tag{5}$$

where wgc is the number of times to defeat the opponent (or the sum of a score in games); tgc is the number of total times to play against an opponent, and tg is artificially preset. The form of the fitness function is an important part of REA-DD. The difficulty

Algorithm 1: REA-DD/RMOEA-DD

Input: EA-Parameters and RL-Parameters
Output: The best population
Initialization(P)
foreach $ind \in P$ **do**
  Use the RL algorithm to train ind;
  Test and obtain the performance of ind;
  Use objective function to evaluate ind;
end
while *stopping criterion not met* **do**
  Use mating selection to select individuals to MP;
  Apply crossover to MP to generate offspring OP;
  foreach $ind \in OP$ **do**
    Use the RL algorithm to train ind;
    Test and obtain the performance of ind;
    Use a fitness function to evaluate ind;
  end
  $PP = P \cup OP$;
  P = Environmental Selection (PP);
end

of game AI to construct a fitness function because it can directly reflect the level of players. The tournament selection is to select individuals to be parents. The crossover operator is the single-point crossover to swap the weights of each channel of the parents to form new individual set OP. The mutation operator is to use Gaussian distribution to change the weight of the individual producing by the crossover operator. To make the weight of the new individual more effective, RL (AC) is used to train and update its weight. The parent population P and the new individual set OP are merged into new set PP. The environmental selection uses to keep the best individuals from PP into the offspring.

The advantages of REA-DD are as follows: (1) it offers a framework of the algorithm that generates DLAI with a different target; (2) it has great accuracy in controlling game DLAI. However, it takes a relatively high cost of environmental frames to generate DLAI. RMOEA-DD will alleviate this problem.

4.2 RMOEA-DD

RMOEA-DD based on RL and MOEA is proposed to improve the efficiency in generating DLAI, which can obtain DLAI at one run. The objective function is defined by formula :

$$f_i = -|\frac{wgc}{tgc} - bd_i|, \tag{6}$$

where $i \in \{1, 2\}$, f_1 and f_2 are conflict with different targets (bd_1 is different from bd_2); wgc and tgc are the same as it in REA-DD.

In RMOEA-DD, we set two objective functions, which represent the upper and lower bounds of DLAI. We could use the objective functions to control the range of DLAI. Generally, DLAI are uniformly distributed in the corresponding range. Thus, we could obtain DLAI by a consistent interval between each target in the range. This is also the best distribution in the range. Therefore we can through pursuing distribution replace of pursuing high precision. This situation is in line with a core issue of MOEA which needs to consider that the diversity and distribution of the population. For this reason, in RMOEA-DD, we just focus on the diversity and distribution of the algorithm. To maintain its distribution and diversity, we introduced the crowded distance operator from NSGA-II, because it has been proved good performance in maintaining distributed and diversity.

Compared with REA-DD which pursues only one goal after setting a goal, RMOEA-DD is a very efficient algorithm. At the same time, in MOEA, good diversity contributes to the convergence of the algorithm. The main process of RMOEA-DD algorithm same as REA-DD shown in Algorithm 1. A difference from REA-DD is environmental selection. In environmental selection, REA-DD retains the highest fitness individuals. In RMOEA-DD, it uses the crowded distance to keep individuals.

The advantages of RMOEA-DD are as follows: (1) it provides an effective framework to the DLAI problem; (2) it reduces greatly the speed of generating DLAI; (3) it lays foundation for control of DLAI using multi-objective optimization in the future.

5 EXPERIMENTS AND ANALYSIS

In this section, we will discuss the results of REA-DD and RMOEA-DD experiment on game *Pong* and *Ghost*.

5.1 Setting and Hyperparameters

Experiment on the *Pong* and *Ghost* , the win rate is used to represent the difficulty level of the game AI. We use 10 different win rates (0.1, 0.2, .., 1) to divide different game difficulty. The higher the difficulty level, the higher the win rate, for example, level 10 represents a 100 % win rate. In *Pong* and *Ghost*, the score or win rate is used to represent the difficulty level of the game AI because it reflects directly to the level of the player and easy to obtain. To better show the level of the player, the score or win rate is obtained after 10 or 30 games. In the contest, the game AI will obtain one score after defeating the opponent. In the battle, the game AI will obtain 1 point after defeating the opponent, otherwise, it will not score.

There are also many parameters in the algorithms. Here we just emphasize some important parameters of EA and RL. In the EA, the population size is 10 trained using 10 CPU cores with the asynchronous methods. The mutation probability is 0.1 and the crossover probability is 1. In the RL algorithm, the learning rate is 0.00025 and training step-size is 10 from REA-DD and RMOEA-DD. In the RL (A3C), the algorithm was trained using 16 CPU cores and the store of the interval of the policy of game AI to 45, which is the best value we can tune out at present in terms of environmental frame overhead.

5.2 Pong

The game *Pong* from Atari, which is a special case of MDP. On each game *Pong* , the player will get a score, when the opponent cannot hit the ball. Each game with no fixed time, the game will end with 21 points, and the player obtains 21 points will win. In *Pong*, we obtain DLAI by controlling the score of agent.

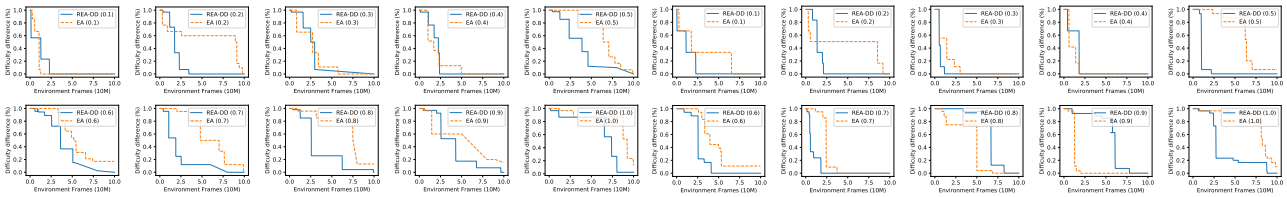


Figure 3: Experiment on the *Pong* (First and second row) and *Ghost* (Third and fourth row): Ten illustrations: REA-DD and EA experiment on ten targets (0.1, 0.2,.., 1). Each curve is the value of best runs based on the difficulty difference. The algorithmic convergence of REA-DD is more excellent than EA.



Figure 4: A screenshot of the scene of the battle in Ghost.

	Single solution		Multiple solution		
Level	REA-DD	EA	RMOEA-DD	MOEA	RL (A3C)
1	**2.282**	6.538	**0.401**	1.199	3.876
2	**2.118**	9.206	**1.046**	3.021	4.423
3	**1.154**	3.019	**0.401**	0.550	3.528
4	1.819	**1.815**	**2.143**	5.191	3.621
5	**2.1891**	11.745	**0.697**	1.683	5.963
6	**4.122**	13.938	**1.871**	2.399	4.657
7	**3.812**	4.121	**1.025**	2.256	3.445
8	8.253	**6.867**	**0.562**	2.352	3.427
9	7.815	**1.946**	**0.720**	2.328	3.590
10	**8.830**	11.870	**2.638**	4.294	3.677

Table 1: Experiment on *Ghost*: The table description of the details of two categories. The first category is REA-DD and EA. The environmental frames of REA-DD are lower than EA. The second category is RMOEA-DD, MOEA, and RL. The environmental frames of RMOEA-DD are lower than MOEA and RL in all targets based on difficulty difference. Note: The unit of the frame based on a million.

5.3 The Ghost Story

The algorithms experiment on the battle scene of *Ghost* which is also solved by approximate modeling into MDPs. Popular Ghost is developed by the Netease company. The battle scene is shown in Figure 4. In the battle scene of *Ghost*, both sides can have their skills, such as a normal attack, heal, exhaust, and masterstroke, etc. Each contest has a fixed time. The condition of winning a game is to kill the opponent or have more blood than the opponent before the game over. Here, some skills have a longer cooling time and some have shorter cooling times. Facing the situation, it is important to release the appropriate skills in the right state for the player to defeat the opponent. The experiment in the form of a 1v1 battle to the built-in high-level player.

5.4 Indicator and Category of REAs

The difficulty difference (or total difficulty difference) and the environmental frames to measure the performance of the algorithms. We calculate the game environmental frames when the difficulty difference (or total difficulty difference) to the specific target. The good performance of the algorithm would have low environmental frames. For the sake of fairness, to better reflect the performance of different algorithms, we divide the algorithms into two categories. The first category: the algorithms (EA and REA-DD) only produce one solution in one run and compared with each other; the second category (MOEA, RMOEA-DD, and RL): the algorithms can generate multiple solutions in one run and compared with each other. The RL algorithm is an A3C algorithm, an asynchronous AC from RMOEA-DD. In RL (A3C), we stored its history policy in running. It also can obtain all DLAI in one run. So we put it into the second category. In the algorithm, we evaluated the speed of the convergence of the algorithm under certain conditions (tolerable overhead). The algorithmic environmental frames are recorded when it converges to the target.

5.5 REA-DD

It could generate DLAI through the different targets in REA-DD. In the experiment, we need to run ten times with different targets. The experiment compares to EA, in which the results are displayed in Figure 3. From Figure 3, we can know that the convergence of REA-DD is better than EA. Experiment on *Pong*, as shown in Figure 3 (first and second rows), the convergence of REA-DD is much better than EA. Experiment on *Ghost*, as shown in Figure 3 (third and fourth rows), the level 5 and the level 9, REA-DD only takes about 2M frames, and the difficulty difference converges to zero. However, it costs about 8M or more in EA. The more details follow Table 1. This phenomenon indicates REA-DD overcomes the deficiency that eliminates some potential individuals. It has a good performance after combining it with RL.

5.6 RMOEA-DD

In RMOEA-DD, we set the values of objective functions to 0 and 1 in the experiment. From the range, we could obtain DLAI in one run. To better show the experimental performance, we compare it

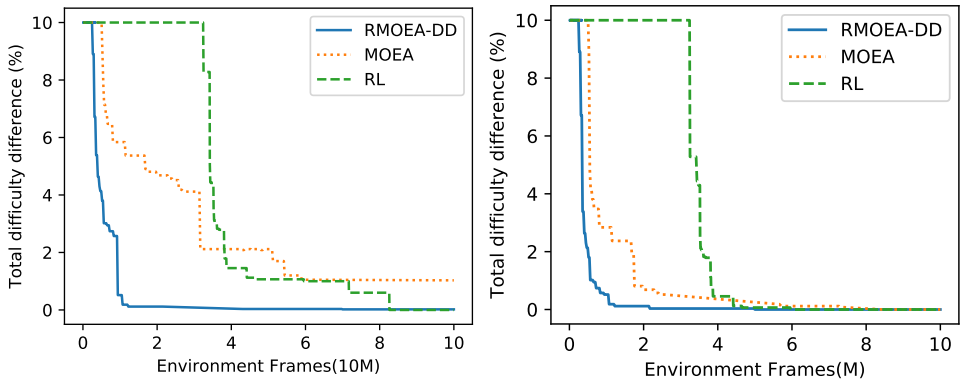


Figure 5: Experiment on *Pong* (the top) and *Ghost* (the bottom): The illustrated description of the comparison of RMOEA-DD, MOEA, and RL. Each curve is the value of best runs based on the total difficulty difference. RMOEA-DD achieves better performance than MOEA and RL.

with MOEA and RL. The algorithmic convergence is evaluated by minimizing the total difficulty difference (TDD). TDD is calculated by the formula:

$$TDD = \sum_{i=1}^{n} \frac{|dif f_i - tg_i|}{tg_i} \qquad (7)$$
$$tg_i \in (0, 1]$$

$dif f_i = \frac{wgc}{tgc}$, wgc and tgc are the same as formula 5; tg_i is from 0 to 1 ($i = 1, tg_i = 0.1$; $i = 2, tg_i = 0.2$; and so on). In the experiments, a good convergence with the lower TDD. The results of the experiments as shown in Figure 5. As Figure 5 shows, the convergence of RMOEA-DD better than MOEA and RL. It takes approximately 10M frames on *Pong*, 2M frames on *Ghost*, to convergence to zero compared to MOEA and RL (A3C), which need to take approximately 80M frames on *Pong*, 4M frames on *Ghost*. It proves that RL has played an important role in the algorithmic convergence. Experiment on Ghost, from Table 1, we can know the level 6 and the level 10 is relative difficult to converge. MOEA is shown in the same situation to RMOEA-DD. For the RL (A3C) algorithm, the reward is sparse in *Pong* and *Ghost*. In *Pong* , the game AI has not found the right direction in the early stage. In *Ghost*, the game AI only be rewarded when the opposite side blood loses. It will result in the game AI does some unnecessary learning in the early stages of the game. Another reason is that the win rate of the game AI sometimes goes up suddenly and it is relatively difficult to decline in RL. In EA, the crossover and mutation can better solve this problem to a certain extent. Compared with REA-DD, we learned that the convergence in RMOEA-DD is better than REA-DD. The main reason is that diversity relatively poor in REA-DD. It can be seen that the algorithmic diversity plays an important role in the speed of the algorithmic convergence in EA. It can be seen that in this paper, the combination of evolutionary algorithm and reinforcement learning is better than the use of a single algorithm. So it is necessary to combine reinforcement learning and evolutionary algorithm reasonably and effectively.

6 CONCLUSIONS AND FUTURE WORK

In this paper, we have provided a framework to the DLAI problem based on RL and EA. In the framework, we presented two algorithms, i.e., REA-DD and RMOEA-DD. Our results show that in our proposed framework the algorithms can efficiently obtain DLAI. Especially, in RMOEA-DD, it uses the idea of multiple objectives to reduce a lot of frames. Currently, we offer DLAI to adjust the difficulty of the game. In EA, we find that diversity is very helpful for algorithm convergence. Based on the framework of REA, we will use it to do more things and explore more potential. In the future, we hope to extend them to more AI design applications.

REFERENCES

[1] Thomas Back. 1996. *Evolutionary algorithms in theory and practice: evolution strategies, evolutionary programming, genetic algorithms*. Oxford university press.
[2] Kalyanmoy Deb. 2001. *Multi-objective optimization using evolutionary algorithms*. Vol. 16. John Wiley & Sons.
[3] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and TAMT Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE transactions on evolutionary computation* 6, 2 (2002), 182–197.
[4] Frank G Glavin and Michael G Madden. 2018. Skilled Experience Catalogue: A Skill-Balancing Mechanism for Non-Player Characters using Reinforcement Learning. In *2018 IEEE Conference on Computational Intelligence and Games (CIG)*. IEEE, 1–8.
[5] Makoto Ishihara, Suguru Ito, Ryota Ishii, Tomohiro Harada, and Ruck Thawonmas. 2018. Monte-Carlo Tree Search for Implementation of Dynamic Difficulty Adjustment Fighting Game AIs Having Believable Behaviors. In *2018 IEEE Conference on Computational Intelligence and Games (CIG)*. IEEE, 1–8.
[6] Vijay R Konda and John N Tsitsiklis. 2000. Actor-critic algorithms. In *Advances in neural information processing systems*. 1008–1014.
[7] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2015. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971* (2015).
[8] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. 2016. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*. 1928–1937.
[9] OpenAI. 2018. OpenAI Five. https://blog.openai.com/openai-five/.
[10] Marc Ponsen, Héctor Munoz-Avila, Pieter Spronck, and David W Aha. 2006. Automatically generating game tactics through evolutionary learning. *AI Magazine* 27, 3 (2006), 75–75.
[11] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. 2017. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864* (2017).
[12] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. 2016. Mastering the game of Go with deep neural

networks and tree search. *Nature* 529, 7587 (2016), 484.

[13] Kenneth O Stanley, Bobby D Bryant, and Risto Miikkulainen. 2005. Real-time neuroevolution in the NERO video game. *IEEE transactions on evolutionary computation* 9, 6 (2005), 653–668.

[14] Alexander L Strehl, Lihong Li, and Michael L Littman. 2009. Reinforcement learning in finite MDPs: PAC analysis. *Journal of Machine Learning Research* 10, 11 (2009), 2413–2444.

[15] Richard S Sutton and Andrew G Barto. 2018. *Reinforcement learning: An introduction*. MIT press.

[16] Richard S Sutton, David A McAllester, Satinder P Singh, and Yishay Mansour. 2000. Policy gradient methods for reinforcement learning with function approximation. In *Advances in neural information processing systems*. 1057–1063.

[17] Chin Hiong Tan, Kay Chen Tan, and Arthur Tay. 2011. Dynamic game difficulty scaling using adaptive behavior-based AI. *IEEE transactions on Computational Intelligence and AI in Games* 3, 4 (2011), 289–301.

[18] John N Tsitsiklis. 1994. Asynchronous stochastic approximation and Q-learning. *Machine learning* 16, 3 (1994), 185–202.

[19] Hado Van Hasselt, Arthur Guez, and David Silver. 2016. Deep reinforcement learning with double Q-learning. In *Thirtieth AAAI conference on artificial intelligence*.

[20] Christopher John Cornish Hellaby Watkins. 1989. Learning from delayed rewards. King's College, Cambridge.

[21] Georgios N Yannakakis, John Levine, and John Hallam. 2004. An evolutionary approach for interactive computer games. In *Proceedings of the 2004 Congress on Evolutionary Computation (IEEE Cat. No. 04TH8753)*, Vol. 1. IEEE, 986–993.

[22] Qingfu Zhang and Hui Li. 2007. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE transactions on evolutionary computation* 11, 6 (2007), 712–731.

[23] Eckart Zitzler, Marco Laumanns, and Lothar Thiele. 2001. SPEA2: Improving the strength Pareto evolutionary algorithm. *TIK-report* 103 (2001).

[24] Mohammad Zohaib. 2018. Dynamic Difficulty Adjustment (DDA) in Computer Games: A Review. *Advances in Human-Computer Interaction* 2018 (2018).